

Experiment-1

```
def encrypt(message, key):
    result=""
    for c in message:
        if c.isupper():
            result+=chr((ord(c)-65+key)%26+65)
        elif c.islower():
            result+=chr((ord(c)-97+key)%26+97)
        else:
            result+=c
    return result
```

```
def decrypt(message, key):
    result=""
    for c in message:
        if c.isupper():
            result+=chr((ord(c)-65-key)%26+65)
        elif c.islower():
            result+=chr((ord(c)-97-key)%26+97)
        else:
            result+=c
    return result
```

```
message=input("Enter the message: ")
key=int(input("Enter the key "))
encrypted=encrypt(message, key)
decrypted=decrypt(encrypted, key)
print("Original message: ", message)
print("Encrypted message: ", encrypted)
print("Decrypted message: ", decrypted)
```

Experiment-2

```
import numpy as np
```

```
def text_to_numbers(text):
    return [ord(c) - 65 for c in text]
```

```
def numbers_to_text(numbers):
    return "".join(chr(int(n) + 65) for n in numbers)
```

```
def mod_inverse(a, m):
    a = a % m
    for i in range(1, m):
        if (a * i) % m == 1:
```

```
        return i
    return None
```

```
def matrix_inverse_mod26(matrix):
    det = int(round(np.linalg.det(matrix)))
    det_mod = det % 26
    det_inv = mod_inverse(det_mod, 26)

    if det_inv is None:
        raise ValueError("Matrix is not invertible")

    adjugate = np.round(det * np.linalg.inv(matrix)).astype(int)
    return (det_inv * adjugate) % 26
```

```
def encrypt(plaintext, key):
    plaintext = plaintext.replace(" ", "").upper()
```

```
    while len(plaintext) % 3 != 0:
        plaintext += 'X'
```

```
    nums = text_to_numbers(plaintext)
    cipher = []
```

```
    for i in range(0, len(nums), 3):
        block = np.array(nums[i:i+3])
        encrypted = key.dot(block) % 26
        cipher.extend(encrypted)
```

```
    return numbers_to_text(cipher)
```

```
def decrypt(message, key):
    key_inv = matrix_inverse_mod26(key)
    nums = text_to_numbers(message)
    plaintext = []
```

```
    for i in range(0, len(nums), 3):
        block = np.array(nums[i:i+3])
        decrypted = key_inv.dot(block) % 26
        plaintext.extend(decrypted)
```

```
    return numbers_to_text(plaintext)
```

```
key_array = np.array([
    [6, 24, 1],
```

```

    [13, 16, 10],
    [20, 17, 15]
])

test = "HELLO"
ciphertext = encrypt(test, key_array)
decrypted_message = decrypt(ciphertext, key_array)

print("Original:", test)
print("Encrypted:", ciphertext)
print("Decrypted:", decrypted_message)

```

Experiment-3

```

def rail_fence_encrypt(text, depth):
    text=text.replace(" ", "").lower()
    rails=[" " for _ in range(depth)]
    row=0
    direction=1
    for c in text:
        rails[row]+=c
        row+=direction
        if row==0 or row==depth-1:
            direction*=-1
    return "".join(rails).upper()

def rail_fence_decrypt(cipher,depth):
    n=len(cipher)
    pattern=[["\n" for _ in range(n)] for _ in range(depth)]
    row=0
    direction=1
    for col in range(n):
        pattern[row][col]='*'
        row+=direction
        if row==0 or row==depth-1:
            direction*=-1

    index=0
    for i in range(n):
        for j in range(depth):
            if pattern[i][j]=='*' and index<n:
                pattern[i][j]=cipher[index]
                index+=1

    result=" "

```

```

row=0
direction=1
for col in range(n):
    result+=pattern[row][col]
    row+=direction
    if row==0 or row==depth-1:
        direction*=-1
return results.upper()

```

```

text="meet me after toga party"
encrypted=rail

```

Experiment-4

```

def encrypt_columnar(plaintext, key):
    plaintext=plaintext.replace(" ", "").lower()
    cols=len(key)
    rows=(len(plaintext)+cols-1) // cols
    k=0
    matrix=[['x' for _ in range(cols)]for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            if k<len(plaintext):
                matrix[i][j]=plaintext[k]
                k+=1

    ciphertext=""
    for nums in range(1, cols+1):
        col=key.index(str(nums))
        for row in range(rows):
            ciphertext+=matrix[row][col]
    return ciphertext.upper()

```

```

def decrypt_columnar(ciphertext, key):
    ciphertext=ciphertext.lower()
    cols=len(key)
    rows=len(ciphertext) // cols
    matrix=[['' for _ in range(cols)]for _ in range(rows)]
    k=0
    for nums in range(1, cols+1):
        col=key.index(str(nums))
        for row in range(rows):
            matrix[row][col]=ciphertext[k]
            k+=1

```

```

plaintext=""
for i in range(rows):
    for j in range(cols):
        plaintext+=matrix[i][j]

return plaintext.rstrip('x').upper()

key="4312567"
text="attack postponed until two am"
ciphertext=encrypt_columnar(text,key)
decrypted=decrypt_columnar(ciphertext, key)

print("Plaintext: ", text)
print("Encrypted:", ciphertext)
print("Decrypted: ", decrypted)

```

Experiment-5a

```

P10 = [3, 5, 2, 7, 4, 10, 1, 9, 8, 6]
P8 = [6, 3, 7, 4, 8, 5, 10, 9]

```

```

def permute(key, table):
    return "".join([key[i-1] for i in table])

def left_shift(bits, shifts):
    return bits[shifts:] + bits[:shifts]

def generate_keys(key10):
    print("Original 10-bit key:", key10)
    key = permute(key10, P10)
    print("After P10 :", key)
    left = key[:5]
    right = key[5:]
    print("Left Half :", left)
    print("Right Half :", right)
    left = left_shift(left, 1)
    right = left_shift(right, 1)
    print("After LS-1 :", left + right)
    key1 = permute(left + right, P8)
    print("Key1 (8-bit):", key1)
    left = left_shift(left, 2)
    right = left_shift(right, 2)
    print("After LS-2 :", left + right)
    key2 = permute(left + right, P8)
    print("In Final Generated Key:", key2)

```

```
return key1, key2
```

```
key_input = input("Enter 10-bit binary key: ")
k1, k2 = generate_keys(key_input)
print("Key1 =", k1)
print("Key2 =", k2)
```

Experiment-5b

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
import base64
```

```
key = b'8bytekey'
```

```
def encrypt(message):
    cipher = DES.new(key, DES.MODE_CBC)
    padded_text = pad(message.encode(), DES.block_size)
    ciphertext = cipher.encrypt(padded_text)
    return base64.b64encode(cipher.iv + ciphertext).decode()
```

```
def decrypt(encrypted_message):
    data = base64.b64decode(encrypted_message)
    iv = data[:8]
    ciphertext = data[8:]
    cipher = DES.new(key, DES.MODE_CBC, iv)
    decrypted = unpad(cipher.decrypt(ciphertext), DES.block_size)
    return decrypted.decode()
```

```
if __name__ == "__main__":
    message = input("Enter message to encrypt: ")
    encrypted_message = encrypt(message)
    print("Encrypted :", encrypted_message)
    decrypted = decrypt(encrypted_message)
    print("Decrypted :", decrypted)
```

Experiment-6a

```
# AES S-box (16x16)
```

```
sbox = [
    [0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76],
    [0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0],
    [0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15],
    [0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75],
```

```
[0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84],
[0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf],
[0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8],
[0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2],
[0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73],
[0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb],
[0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79],
[0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08],
[0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a],
[0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e],
[0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf],
[0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16]
]
```

```
# Initial state matrix
```

```
state = [
    [0x19, 0xa0, 0x9a, 0xe9],
    [0x3d, 0xf4, 0xc6, 0xf8],
    [0xe3, 0xe2, 0x8d, 0x48],
    [0xbe, 0x2b, 0x2a, 0x08]
]
```

```
# Function to print matrix nicely
```

```
def print_state(s):
    for row in s:
        print(['{:02x}'.format(x) for x in row])
    print()
```

```
# SubBytes step
```

```
def sub_bytes(state):
    for i in range(4):
        for j in range(4):
            byte = state[i][j]
            row = byte >> 4 # upper 4 bits
            col = byte & 0x0F # lower 4 bits
            state[i][j] = sbox[row][col]
    return state
```

```
# ShiftRows step
```

```
def shift_rows(state):
    state[1] = state[1][1:] + state[1][:1]
    state[2] = state[2][2:] + state[2][:2]
    state[3] = state[3][3:] + state[3][:3]
    return state
```

```
# ---- Main Execution ----  
print("Original State Matrix:")  
print_state(state)
```

```
state = sub_bytes(state)  
print("After SubBytes:")  
print_state(state)
```

```
state = shift_rows(state)  
print("After ShiftRows:")  
print_state(state)
```

Experiment-6b

```
from Crypto.Cipher import AES  
from Crypto.Util.Padding import pad, unpad  
import base64
```

```
key = b'16bytekey1234567' # AES requires 16, 24, or 32 bytes
```

```
def encrypt(message):  
    cipher = AES.new(key, AES.MODE_CBC)  
    padded_text = pad(message.encode(), AES.block_size)  
    ciphertext = cipher.encrypt(padded_text)  
    return base64.b64encode(cipher.iv + ciphertext).decode()
```

```
def decrypt(encrypted_message):  
    data = base64.b64decode(encrypted_message)  
    iv = data[:16] # AES IV is 16 bytes (DES was 8)  
    ciphertext = data[16:]  
    cipher = AES.new(key, AES.MODE_CBC, iv)  
    decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)  
    return decrypted.decode()
```

```
if __name__ == "__main__":  
    message = input("Enter message to encrypt: ")  
    encrypted_message = encrypt(message)  
    print("Encrypted :", encrypted_message)  
    decrypted = decrypt(encrypted_message)  
    print("Decrypted :", decrypted)
```